

TECHNICAL DESIGN DOCUMENT

World Cup Intelligence Platform

Data model, predictive engine, and AI layer

Author Tobi Smith-Kayode

Role Software Engineer / Data Scientist

Version 1.0

Status Implemented

Date June 2026

Portfolio artifact — based on a real, production-grade system designed and built by the author.

Contents

- 1. Summary
- 2. Architecture Overview
- 3. Domain Model
- 4. Data Ingestion & Adapters
- 5. The Analytics Engine
- 6. Caching & Lifecycle
- 7. The AI Layer
- 8. Key Technical Decisions
- 9. Testing & Verification
- 10. Failure Modes & Resilience
- Appendix A — Technology Stack

1. Summary

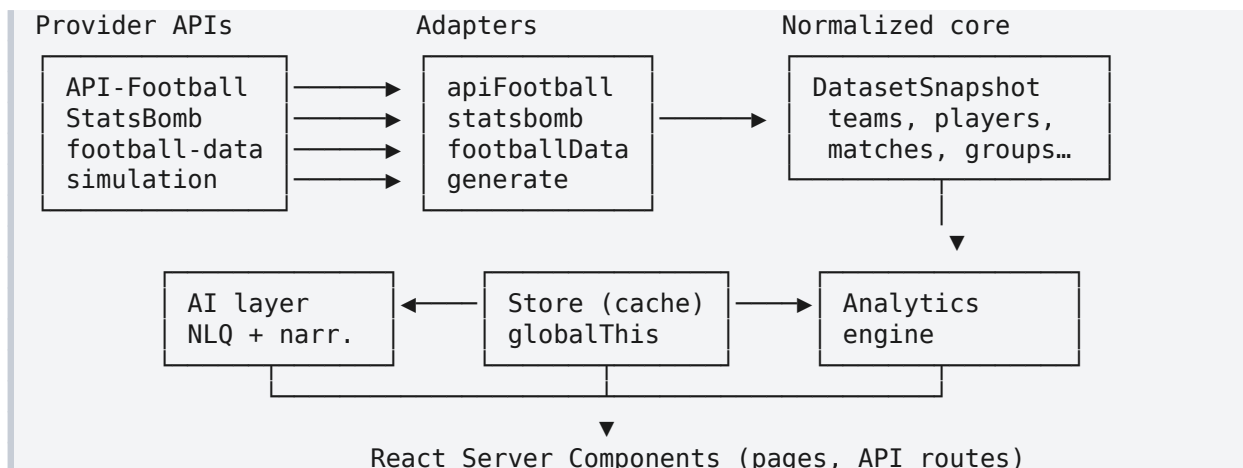
This document describes the technical design of the World Cup Intelligence Platform: a server-rendered analytics application that ingests football data from multiple sources, normalizes it into a single domain model, runs a probabilistic forecasting engine over it, and exposes the results through interactive pages and an AI query/narrative layer.

The system is built as a **Next.js 14 (App Router)** application in TypeScript. It deliberately has **no separate backend or database** at rest: data is fetched from providers, normalized in-process, cached, and served by React Server Components. The interesting engineering is concentrated in three places — the **data-normalization layer**, the **analytics engine**, and the **AI layer** — which this document covers in depth.

Design goal One analytical core that is completely agnostic to where the data came from or which tournament is active. Adding a new data source or a new tournament must not touch the engine, the pages, or the AI layer.

2. Architecture Overview

Data flows in one direction: providers → adapters → a normalized snapshot → an in-memory store → the analytics engine → server-rendered views and the AI layer.



3. Domain Model

Every adapter targets one shared type, `DatasetSnapshot`. This is the contract that makes the rest of the system source-agnostic: the engine, pages and AI layer only ever see a snapshot, never a provider payload.

Entity	Key fields	Notes
Competition	id, name, format (group sizes, knockout shape)	Drives format-agnostic bracketing
Team	id, code, confederation, groupId, elo,	ELO seeded from rating

Entity	Key fields	Notes
	attack/defenseRating, squadIds	priors
Player	id, teamId, position, detailedPosition, shirtNumber, age	id is namespaced: teamCode-providerId
PlayerStats	goals, assists, xG, xA, per-90 derivations, percentiles	Computed lazily into a PlayerView
Match	homeTeamId, awayTeamId, stage, status, score, events, teamStats	Supports LIVE / SCHEDULED / FINISHED
Group	id, teamIds	Inferred when the source doesn't label groups

Design decision — namespaced IDs Player IDs are stored as teamCode-providerId (e.g. arg-19599). Because API-Football reuses a player's ID across its club and country endpoints, this scheme lets us join a World Cup squad to club rosters by extracting the provider ID — the mechanism behind the Club Connections feature.

4. Data Ingestion & Adapters

Each source has a single adapter whose only job is to return a DatasetSnapshot. Adapters absorb every provider quirk so nothing downstream has to know the difference.

4.1 Adapter responsibilities

- **Field mapping** — translate provider fields to the domain model (e.g. API-Football's broad role → GK/DF/MF/FW, with a representative detailed position rather than a fabricated one).
- **Reconstruction** — derive structure the source omits. StatsBomb's open data doesn't label group-stage groups, so the adapter infers them by clustering teams that share group-stage opponents.
- **Resilience** — tolerate partial responses and provider rate-limits without failing the whole load (see §10).

4.2 The live adapter (API-Football)

The live path is the most demanding: a full bootstrap fans out to per-team squad, coach, standings and paginated player-stats endpoints — roughly 150 calls. To stay within rate limits it:

- Batches squad/coach requests (≤ 5 per batch) with inter-batch pacing.
- **Retries on rate-limit** — both HTTP 429 and API-Football's body-level error object (it sometimes returns 200 with an errors map) — using escalating backoff (2s → 10s).
- Caches provider responses for a short window to absorb refetch pressure.

5. The Analytics Engine

The engine is the statistical heart of the product. It takes a snapshot and produces ratings, a calibrated match model, and a full tournament forecast. It is pure and deterministic given its inputs, and is recomputed (and cached) per tournament.

5.1 Team strength — ELO with margin of victory

Each team carries an ELO rating, seeded from rating priors and updated over observed results. The update is the standard ELO step, scaled by margin of victory so emphatic wins move ratings more than narrow ones:

```
Expected score:  E_home = 1 / (1 + 10^(-(R_home + H - R_away) / 400))
Update:          R'      = R + K · g(Δ) · (S - E)
  where  S  = actual result (1 / 0.5 / 0)
         H  = home advantage (rating points)
         K  = learning rate
         g(Δ) = margin-of-victory multiplier, increasing in goal difference Δ
```

The ELO gap between two teams maps directly to a win expectancy, which is what the knockout simulation uses to resolve ties.

5.2 Match model — bivariate Poisson

Score lines are modeled with a **bivariate-Poisson** model. Each team's expected goals in a fixture is a function of its attack strength, the opponent's defensive strength and home advantage; a shared component captures the positive correlation between the two teams' scores (open, end-to-end games inflate both tallies).

```
X = goals_home,  Y = goals_away
X = X1 + X3,    Y = X2 + X3,    Xi ~ Poisson(λi)

log λ1 = μ + home + att_home + def_away      (home expected goals)
log λ2 = μ +      att_away + def_home        (away expected goals)
λ3 ≥ 0  couples the two scores (draw inflation / shared game-state)
```

From the joint distribution over (X, Y) the model derives win / draw / loss probabilities and a full scoreline distribution for any fixture — the numbers shown on every match card.

Why bivariate, not two independent Poissons Independent Poisson models systematically under-predict draws. The shared λ_3 term restores the observed draw mass, which materially changes group-stage qualification math where draws are decisive.

5.3 Tournament forecast — Monte Carlo

The forecast is produced by **simulating the remainder of the tournament 8,000 times**. Each run is a complete playthrough:

1. Complete the group stage by sampling every unplayed fixture from the match model.
2. Rank each group and resolve the best third-placed teams (when the format requires it).
3. Seed the knockout bracket using the competition's actual format (group sizes and knockout shape are read from the snapshot, not hard-coded).
4. Play out every knockout tie via ELO win expectancy, sampling the winner.

Aggregating across the 8,000 runs yields, for every team, the probability of winning its group, advancing, reaching each round, and lifting the trophy. Because the output is a distribution rather than a single bracket, the UI can show **how often** an outcome occurs rather than a brittle single prediction.

Output	Definition
Title %	Share of simulations in which the team won the tournament
Stage-reach %	Share reaching R16 / QF / SF / Final (monotonically decreasing)
Group-win %	Share finishing first in the group
Expected finish	Mean final placement across runs
Golden Boot %	Share in which a given player finished top scorer

5.4 Derived metrics

- **Power ratings** — a blended attack/defense strength score used for ranking.
- **Golden Boot projection** — current goals regressed toward xG, then projected forward over each player's remaining expected minutes, so hot/cold finishing reverts to chance quality.
- **Player percentiles** — per-90 metrics ranked within positional cohorts to power the radar charts and scouting reports.

6. Caching & Lifecycle

Because there is no database, the normalized snapshot and the computed engine are the application's hot state. Both are cached on `globalThis`, deliberately.

Design decision — `globalThis` cache Next.js runs the startup instrumentation hook in a different module instance from request handlers, so a plain module-level variable set at boot is invisible to requests. Pinning the snapshot, the per-tournament cache and the computed engine to `globalThis` gives every execution context one shared source of truth.

- **Background load** — the startup hook kicks off the live fetch without blocking, so the server is responsive immediately and serves a deterministic fallback dataset until the live snapshot is ready.
- **Multi-tournament cache** — snapshots are keyed by tournament ID; switching tournaments swaps the active key and clears the derived engine cache so it recomputes for the new dataset.
- **Invalidation** — setting a new dataset clears the engine and all secondary indexes in one place, preventing stale-engine/fresh-data mismatches.

7. The AI Layer

Two AI-flavored subsystems sit on top of the normalized data; both are designed to be useful with zero external dependencies and to upgrade gracefully when a hosted LLM is available.

7.1 Natural-language query engine

The "Ask" feature accepts plain-English questions ("who leads the golden boot?", "compare France and Brazil"). A lightweight intent parser tokenizes the query, classifies it (leaderboard, comparison, team lookup, forecast, breakout), resolves the entities and metric, and dispatches to a typed handler that answers **directly from the snapshot**. Answers are structured (a sentence plus a small table), so they render consistently and never hallucinate a statistic.

7.2 Narrative & insight generation

Insights ("Argentina defying the odds"), the daily briefing, player storylines and match summaries are produced by a deterministic generator that reads the engine output and selects archetypes (upset, breakout, momentum mover, golden-boot leader) with templated, data-filled prose.

A thin `narrate()` wrapper can upgrade any of these to LLM-authored prose when an API key is configured, passing the structured payload to a hosted model — but the deterministic generator is the always-on fallback, so the product never requires network access or a key to produce readable narratives.

AI design principle The model never invents facts. Numbers are computed deterministically from data; the LLM (when present) only rewrites a pre-computed, structured payload into prose. This keeps every claim verifiable.

8. Key Technical Decisions

Decision	Alternatives considered	Rationale
No database	Postgres / SQLite cache	Data is provider-owned and refreshes constantly; in-process normalization + cache is simpler and stateless to deploy.
globalThis state	Module singletons; external cache (Redis)	Survives Next's multi-instance module loading without extra infra.
Force-dynamic rendering	Static generation / ISR	Live data must never be baked at build time; every analytical route renders on demand.
Adapter + snapshot	Per-source code paths in pages	Keeps the engine, pages and AI layer 100% source-agnostic.
Deterministic AI core	LLM-first narration	Verifiable, offline-capable, zero-cost; LLM is an optional upgrade, not a dependency.

9. Testing & Verification

- **Unit tests** on the engine — ELO updates, the match model's probability mass, and format-agnostic bracketing across 32- and 48-team shapes.
- **End-to-end data sweeps** — automated checks that every team, in every tournament, resolves a complete squad, coach, standing and forecast; run across all six datasets.
- **Failure-injection** — exercising the app during the live-load window and across rapid tournament switches to confirm zero render exceptions under inconsistent state.

10. Failure Modes & Resilience

The system assumes its inputs are imperfect — providers rate-limit, open data has gaps, and live state is briefly inconsistent during load. Three patterns keep it robust:

5. **Retry with backoff** on the live feed (HTTP- and body-level rate limits) so a transient 429 never silently drops a team's squad.
6. **Lookup guards at the source** — entity resolution returns "absent" rather than asserting non-null, so a player whose team hasn't loaded yet is omitted, never the cause of a crash.
7. **Graceful degradation per surface** — any page renders a sensible empty/loading state for missing data instead of failing the whole view.

Engineering note A whole class of intermittent crashes traced back to non-null assertions on entity lookups that are briefly undefined while live data loads. Fixing this at the resolution layer — return undefined and filter — eliminated the entire class rather than patching each call site.

Appendix A — Technology Stack

Layer	Technology
Framework	Next.js 14 (App Router), React Server Components
Language	TypeScript (strict)
UI	Tailwind CSS, Recharts, react-globe.gl / three.js
Data	API-Football (live), StatsBomb open data (historical), football-data.org
Modeling	Custom ELO, bivariate-Poisson, Monte Carlo (in-process)
AI	Deterministic NLQ + narrative generators; optional hosted-LLM narration